



UTEC Posgrado



UTEC Posgrado

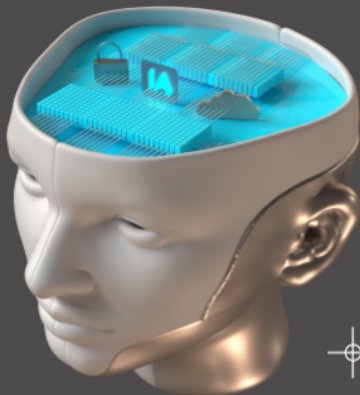
DEEP LEARNING

PRÁCTICA: REDES NEURONALES



Parte 1

Conceptos previos para la práctica



Normalización de datos

¿Por qué normalizar? Las redes neuronales aprenden más rápido y de forma más estable cuando las entradas tienen escala similar.



Normalización de datos

¿Por qué normalizar? Las redes neuronales aprenden más rápido y de forma más estable cuando las entradas tienen escala similar.

Min-Max Normalization:

$$x'_j = \frac{x_j - x_j^{\text{mín}}}{x_j^{\text{máx}} - x_j^{\text{mín}}} \in [0, 1]$$



Normalización de datos

¿Por qué normalizar? Las redes neuronales aprenden más rápido y de forma más estable cuando las entradas tienen escala similar.

Min-Max Normalization:

$$x'_j = \frac{x_j - x_j^{\text{mín}}}{x_j^{\text{máx}} - x_j^{\text{mín}}} \in [0, 1]$$

Regla de oro: La normalización se **ajusta SOLO** con los datos de entrenamiento y se **aplica también** al test. Nunca usar estadísticas del test para normalizar.

	Original	Normalizado
Entrenamiento	$\{x^{(1)}, \dots, x^{(m)}\}$	$x' = \frac{x - x_{\text{train}}^{\text{mín}}}{x_{\text{train}}^{\text{máx}} - x_{\text{train}}^{\text{mín}}}$
Test	$\{x_t^{(1)}, \dots\}$	$x'_t = \frac{x_t - x_{\text{train}}^{\text{mín}}}{x_{\text{train}}^{\text{máx}} - x_{\text{train}}^{\text{mín}}}$



Evaluación del modelo: Accuracy

¿Cómo medir si la red aprendió?



Evaluación del modelo: Accuracy

¿Cómo medir si la red aprendió?

Para clasificación binaria, usamos el **umbral de decisión** $\hat{y} \geq 0,5 \Rightarrow$ clase 1, de lo contrario clase 0:

$$\hat{y}_{\text{pred}} = \begin{cases} 1 & \text{si } \hat{y} \geq 0,5 \\ 0 & \text{si } \hat{y} < 0,5 \end{cases}$$



Evaluación del modelo: Accuracy

¿Cómo medir si la red aprendió?

Para clasificación binaria, usamos el **umbral de decisión** $\hat{y} \geq 0,5 \Rightarrow$ clase 1, de lo contrario clase 0:

$$\hat{y}_{\text{pred}} = \begin{cases} 1 & \text{si } \hat{y} \geq 0,5 \\ 0 & \text{si } \hat{y} < 0,5 \end{cases}$$

La **accuracy** (exactitud) es la fracción de predicciones correctas:

$$\text{Accuracy} = \frac{\text{Número de predicciones correctas}}{\text{Total de ejemplos}} \times 100 \%$$

Importante: Durante el test, **no** se calculan gradientes ni se actualizan pesos.



¿Qué es **PyTorch**? Biblioteca de Python para cómputo tensorial con diferenciación automática (autograd).



¿Qué es **PyTorch**? Biblioteca de Python para cómputo tensorial con diferenciación automática (autograd).

Tensor: arreglo N-dimensional (como numpy pero con GPU y autograd).

```
import torch
x = torch.tensor([[1.0, 2.0]])
W = torch.randn(2, 3, requires_grad=True)
z = x @ W # (1,3)
```



¿Qué es PyTorch? Biblioteca de Python para cómputo tensorial con diferenciación automática (autograd).

Tensor: arreglo N-dimensional (como numpy pero con GPU y autograd).

```
import torch
x = torch.tensor([[1.0, 2.0]])
W = torch.randn(2, 3, requires_grad=True)
z = x @ W # (1,3)
```

Autograd: Calcula gradientes automáticamente.

```
loss = z.sum()
loss.backward() # calcula grad
print(W.grad) #  $\partial \text{loss} / \partial W$ 
```



¿Qué es PyTorch? Biblioteca de Python para cómputo tensorial con diferenciación automática (autograd).

Tensor: arreglo N-dimensional (como numpy pero con GPU y autograd).

Autograd: Calcula gradientes automáticamente.

```
import torch
x = torch.tensor([[1.0, 2.0]])
W = torch.randn(2, 3, requires_grad=True)
z = x @ W # (1,3)
```

```
loss = z.sum()
loss.backward() # calcula grad
print(W.grad) #  $\partial \text{loss} / \partial W$ 
```

Componentes clave:

- ▶ `nn.Module`: clase base para definir redes
- ▶ `nn.Linear(in, out)`: capa lineal $z = xW^T + b$
- ▶ `torch.optim.SGD` / `Adam`: optimizadores
- ▶ `nn.BCELoss()`: Binary Cross-Entropy
- ▶ `model.eval()` + `torch.no_grad()`: modo inferencia



Flujo de trabajo en PyTorch



Flujo de trabajo en PyTorch



Cada época:

1. `optimizer.zero_grad()`
2. `y_hat = model(X)`
3. `loss = criterion(y_hat, y)`
4. `loss.backward()`
5. `optimizer.step()`

Modo evaluación:

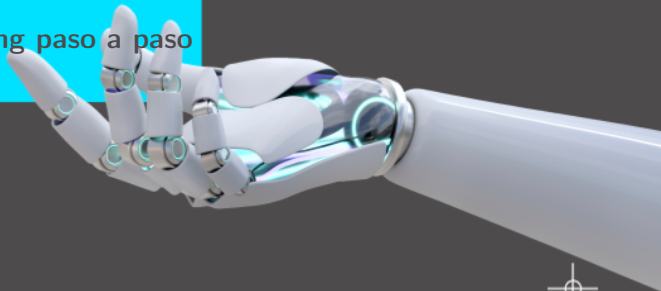
- ```
1. model.eval()
2. with torch.no_grad():
3. y_hat = model(X_test)
4. accuracy = ...
```





# Parte 2

## Ejercicio 1 resuelto Training y Testing paso a paso



# Problema: Diagnóstico de Tumores

**Objetivo:** Clasificar tumores como **benigno** ( $y = 0$ ) o **maligno** ( $y = 1$ ) a partir de dos biomarcadores normalizados.

**Dataset de Entrenamiento** (10 ejemplos):

| $i$ | $x_1$ | $x_2$ | $y$ |
|-----|-------|-------|-----|
| 1   | 0.20  | 0.10  | 0   |
| 2   | 0.30  | 0.30  | 0   |
| 3   | 0.10  | 0.20  | 0   |
| 4   | 0.40  | 0.20  | 0   |
| 5   | 0.30  | 0.40  | 0   |
| 6   | 0.80  | 0.90  | 1   |
| 7   | 0.90  | 0.80  | 1   |
| 8   | 0.70  | 0.90  | 1   |
| 9   | 0.80  | 0.70  | 1   |
| 10  | 0.90  | 0.60  | 1   |

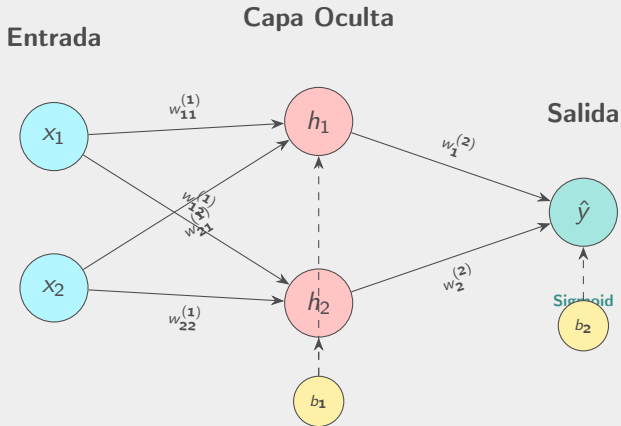
**Dataset de Test** (4 ejemplos):

| $i$ | $x_1$ | $x_2$ | $y$ |
|-----|-------|-------|-----|
| 1   | 0.20  | 0.30  | 0   |
| 2   | 0.40  | 0.10  | 0   |
| 3   | 0.70  | 0.80  | 1   |
| 4   | 0.90  | 0.70  | 1   |

**Nota:** Datos ya normalizados  $\in [0, 1]$ .

Clases 0 (benigno): valores bajos.

Clases 1 (maligno): valores altos.



Arquitectura: **2 entradas**  $\rightarrow$  **2 neuronas ocultas (ReLU)**  $\rightarrow$  **1 salida (Sigmoid)**  
 de parámetros:  $2 \times 2 + 2 + 2 \times 1 + 1 = 9$

# Paso 1: Inicialización de Pesos

**Recordar:** Los pesos se inicializan con valores pequeños. En este ejemplo usamos valores predefinidos para facilitar el cálculo a mano.

**Capa Oculta** ( $W^{(1)}$ ,  $b^{(1)}$ ):

$$W^{(1)} = \begin{bmatrix} 0,5 & 0,3 \\ -0,4 & 0,6 \end{bmatrix}_{2 \times 2}$$

$$b^{(1)} = [0,0 \quad 0,0]_{1 \times 2}$$

$W_{ij}^{(1)}$ : peso de entrada  $i$  a neurona oculta  $j$ .

**Capa de Salida** ( $W^{(2)}$ ,  $b^{(2)}$ ):

$$W^{(2)} = \begin{bmatrix} 0,7 \\ 0,4 \end{bmatrix}_{2 \times 1}$$

$$b^{(2)} = [0,0]_{1 \times 1}$$

**Tasa de aprendizaje:**  $\alpha = 0,5$



# Paso 1: Inicialización de Pesos

**Recordar:** Los pesos se inician con valores pequeños. En este ejemplo usamos valores predefinidos para facilitar el cálculo a mano.

**Capa Oculta** ( $W^{(1)}, b^{(1)}$ ):

$$W^{(1)} = \begin{bmatrix} 0,5 & 0,3 \\ -0,4 & 0,6 \end{bmatrix}_{2 \times 2}$$

$$b^{(1)} = [0,0 \quad 0,0]_{1 \times 2}$$

**Capa de Salida** ( $W^{(2)}, b^{(2)}$ ):

$$W^{(2)} = \begin{bmatrix} 0,7 \\ 0,4 \end{bmatrix}_{2 \times 1}$$

$$b^{(2)} = [0,0]_{1 \times 1}$$

$W_{ij}^{(1)}$ : peso de entrada  $i$  a neurona oculta  $j$ .

**Tasa de aprendizaje:**  $\alpha = 0,5$

**Dimensiones en matrices:** Con  $m = 10$  ejemplos,  $n_x = 2$  entradas,  $n_h = 2$  neuronas ocultas:

$$X \in \mathbb{R}^{10 \times 2}, \quad Z^{(1)}, A^{(1)} \in \mathbb{R}^{10 \times 2}, \quad Z^{(2)}, \hat{Y} \in \mathbb{R}^{10 \times 1}$$



## Paso 2: Forward Pass — Ecuaciones

Flujo de información de entrada a salida:

Capa oculta (cada ejemplo es una fila de  $X$ ):

$$Z^{(1)} = X W^{(1)} + b^{(1)}, \quad (10 \times 2) = (10 \times 2)(2 \times 2) + (1 \times 2)$$

$$A^{(1)} = \text{ReLU}(Z^{(1)}) = \text{máx}(0, Z^{(1)}) \quad (\text{elemento a elemento})$$



## Paso 2: Forward Pass — Ecuaciones

Flujo de información de entrada a salida:

Capa oculta (cada ejemplo es una fila de  $X$ ):

$$Z^{(1)} = X W^{(1)} + b^{(1)}, \quad (10 \times 2) = (10 \times 2)(2 \times 2) + (1 \times 2)$$

$$A^{(1)} = \text{ReLU}(Z^{(1)}) = \text{máx}(0, Z^{(1)}) \quad (\text{elemento a elemento})$$

Capa de salida:

$$Z^{(2)} = A^{(1)} W^{(2)} + b^{(2)}, \quad (10 \times 1) = (10 \times 2)(2 \times 1) + (1 \times 1)$$

$$\hat{Y} = \sigma(Z^{(2)}) = \frac{1}{1 + e^{-Z^{(2)}}} \quad (\text{elemento a elemento})$$



## Paso 2: Forward Pass — Ecuaciones

Flujo de información de entrada a salida:

Capa oculta (cada ejemplo es una fila de  $X$ ):

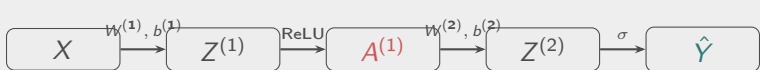
$$Z^{(1)} = X W^{(1)} + b^{(1)}, \quad (10 \times 2) = (10 \times 2)(2 \times 2) + (1 \times 2)$$

$$A^{(1)} = \text{ReLU}(Z^{(1)}) = \max(0, Z^{(1)}) \quad (\text{elemento a elemento})$$

Capa de salida:

$$Z^{(2)} = A^{(1)} W^{(2)} + b^{(2)}, \quad (10 \times 1) = (10 \times 2)(2 \times 1) + (1 \times 1)$$

$$\hat{Y} = \sigma(Z^{(2)}) = \frac{1}{1 + e^{-Z^{(2)}}} \quad (\text{elemento a elemento})$$





## Paso 2: Forward Pass — Ejemplo Numérico ( $i = 3$ )

**Ejemplo 3:**  $x = [0,10, 0,20]$ ,  $y = 0$  (benigno)

**Cálculo de  $Z^{(1)}$ :**

$$z_{h_1}^{(1)} = w_{11}^{(1)} \cdot x_1 + w_{21}^{(1)} \cdot x_2 + b_1^{(1)} = 0,5 \times 0,10 + (-0,4) \times 0,20 + 0 = 0,05 - 0,08 = -0,03$$

$$z_{h_2}^{(1)} = w_{12}^{(1)} \cdot x_1 + w_{22}^{(1)} \cdot x_2 + b_2^{(1)} = 0,3 \times 0,10 + 0,6 \times 0,20 + 0 = 0,03 + 0,12 = 0,15$$



## Paso 2: Forward Pass — Ejemplo Numérico ( $i = 3$ )

**Ejemplo 3:**  $x = [0,10, 0,20]$ ,  $y = 0$  (benigno)

**Cálculo de  $Z^{(1)}$ :**

$$z_{h_1}^{(1)} = w_{11}^{(1)} \cdot x_1 + w_{21}^{(1)} \cdot x_2 + b_1^{(1)} = 0,5 \times 0,10 + (-0,4) \times 0,20 + 0 = 0,05 - 0,08 = -0,03$$

$$z_{h_2}^{(1)} = w_{12}^{(1)} \cdot x_1 + w_{22}^{(1)} \cdot x_2 + b_2^{(1)} = 0,3 \times 0,10 + 0,6 \times 0,20 + 0 = 0,03 + 0,12 = 0,15$$

**Aplicando ReLU:**

$$a_{h_1}^{(1)} = \text{ReLU}(-0,03) = \max(0, -0,03) = 0,00 \quad \leftarrow \text{¡ReLU corta la activación!}$$

$$a_{h_2}^{(1)} = \text{ReLU}(0,15) = 0,15$$



## Paso 2: Forward Pass — Ejemplo Numérico ( $i = 3$ )

**Ejemplo 3:**  $x = [0,10, 0,20]$ ,  $y = 0$  (benigno)

**Cálculo de  $Z^{(1)}$ :**

$$z_{h_1}^{(1)} = w_{11}^{(1)} \cdot x_1 + w_{21}^{(1)} \cdot x_2 + b_1^{(1)} = 0,5 \times 0,10 + (-0,4) \times 0,20 + 0 = 0,05 - 0,08 = -0,03$$

$$z_{h_2}^{(1)} = w_{12}^{(1)} \cdot x_1 + w_{22}^{(1)} \cdot x_2 + b_2^{(1)} = 0,3 \times 0,10 + 0,6 \times 0,20 + 0 = 0,03 + 0,12 = 0,15$$

**Aplicando ReLU:**

$$a_{h_1}^{(1)} = \text{ReLU}(-0,03) = \max(0, -0,03) = 0,00 \quad \leftarrow \text{¡ReLU corta la activación!}$$

$$a_{h_2}^{(1)} = \text{ReLU}(0,15) = 0,15$$

**Cálculo de  $Z^{(2)}$  y  $\hat{y}$ :**

$$z^{(2)} = 0,7 \times 0,00 + 0,4 \times 0,15 + 0 = 0 + 0,060 = 0,060$$

$$\hat{y} = \sigma(0,060) = \frac{1}{1 + e^{-0,060}} \approx \frac{1}{1,942} \approx 0,515$$

Predicción:  $0,515 > 0,5 \Rightarrow$  clase 1. ¡Incorrecto ( $y = 0$ )! La red aún debe aprender.



## Paso 2: Forward Pass — Resultado Completo

Resultado para los 10 ejemplos de entrenamiento:

| $i$ | $x_1$ | $x_2$ | $y$ | $z_{h_1}^{(1)}$ | $z_{h_2}^{(1)}$ | $a_{h_1}^{(1)}$ | $a_{h_2}^{(1)}$ | $z^{(2)}$ | $\hat{y}$ |
|-----|-------|-------|-----|-----------------|-----------------|-----------------|-----------------|-----------|-----------|
| 1   | .20   | .10   | 0   | 0.06            | 0.12            | 0.06            | 0.12            | 0.090     | 0.522     |
| 2   | .30   | .30   | 0   | 0.03            | 0.27            | 0.03            | 0.27            | 0.129     | 0.532     |
| 3   | .10   | .20   | 0   | -0.03           | 0.15            | 0.00            | 0.15            | 0.060     | 0.515     |
| 4   | .40   | .20   | 0   | 0.12            | 0.24            | 0.12            | 0.24            | 0.180     | 0.545     |
| 5   | .30   | .40   | 0   | -0.01           | 0.33            | 0.00            | 0.33            | 0.132     | 0.533     |
| 6   | .80   | .90   | 1   | 0.04            | 0.78            | 0.04            | 0.78            | 0.340     | 0.584     |
| 7   | .90   | .80   | 1   | 0.13            | 0.75            | 0.13            | 0.75            | 0.391     | 0.597     |
| 8   | .70   | .90   | 1   | -0.01           | 0.75            | 0.00            | 0.75            | 0.300     | 0.574     |
| 9   | .80   | .70   | 1   | 0.12            | 0.66            | 0.12            | 0.66            | 0.348     | 0.586     |
| 10  | .90   | .60   | 1   | 0.21            | 0.63            | 0.21            | 0.63            | 0.399     | 0.598     |

Celdas rosadas : ReLU corta la activación ( $z < 0 \Rightarrow a = 0$ ). Todos los  $\hat{y} \approx 0,5$  (inicio de entrenamiento)



## Paso 3: Función de Pérdida BCE

**Recordar:** Binary Cross-Entropy mide qué tan lejos están las predicciones de las etiquetas reales.

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$



## Paso 3: Función de Pérdida BCE

**Recordar:** Binary Cross-Entropy mide qué tan lejos están las predicciones de las etiquetas reales.

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

**Para  $y = 0$ :**  $\mathcal{L}_i = -\log(1 - \hat{y}^{(i)})$

**Para  $y = 1$ :**  $\mathcal{L}_i = -\log(\hat{y}^{(i)})$

**Clase 0** ( $-\log(1 - \hat{y})$ )

**Clase 1** ( $-\log(\hat{y})$ )

| $i = 1$ | $i = 2$ | $i = 3$ | $i = 4$ | $i = 5$ | $i = 6$ | $i = 7$ | $i = 8$ | $i = 9$ | $i = 10$ |
|---------|---------|---------|---------|---------|---------|---------|---------|---------|----------|
| 0.738   | 0.758   | 0.724   | 0.788   | 0.761   | 0.537   | 0.516   | 0.555   | 0.534   | 0.514    |



## Paso 3: Función de Pérdida BCE

**Recordar:** Binary Cross-Entropy mide qué tan lejos están las predicciones de las etiquetas reales.

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

**Para  $y = 0$ :**  $\mathcal{L}_i = -\log(1 - \hat{y}^{(i)})$

**Para  $y = 1$ :**  $\mathcal{L}_i = -\log(\hat{y}^{(i)})$

**Clase 0** ( $-\log(1 - \hat{y})$ )

**Clase 1** ( $-\log(\hat{y})$ )

| $i = 1$ | $i = 2$ | $i = 3$ | $i = 4$ | $i = 5$ | $i = 6$ | $i = 7$ | $i = 8$ | $i = 9$ | $i = 10$ |
|---------|---------|---------|---------|---------|---------|---------|---------|---------|----------|
| 0.738   | 0.758   | 0.724   | 0.788   | 0.761   | 0.537   | 0.516   | 0.555   | 0.534   | 0.514    |

$$\mathcal{L} = \frac{1}{10} (0,738 + 0,758 + 0,724 + 0,788 + 0,761 + 0,537 + 0,516 + 0,555 + 0,534 + 0,514)$$

$$\mathcal{L} = \frac{6,425}{10} = 0,643$$

## Paso 4: Retropropagación — Ecuaciones

**Recordar:** La retropropagación usa la regla de la cadena para calcular  $\nabla \mathcal{L}$  respecto a cada parámetro.

Capa de salida:

$$\delta^{(2)} = \hat{Y} - Y, \quad (m \times 1)$$

$$\nabla_{W^{(2)}} \mathcal{L} = \frac{1}{m} A^{(1)\top} \delta^{(2)}, \quad \nabla_{b^{(2)}} \mathcal{L} = \frac{1}{m} \sum_i \delta_i^{(2)}$$





## Paso 4: Retropropagación — Ecuaciones

**Recordar:** La retropropagación usa la regla de la cadena para calcular  $\nabla \mathcal{L}$  respecto a cada parámetro.

**Capa de salida:**

$$\delta^{(2)} = \hat{Y} - Y, \quad (m \times 1)$$

$$\nabla_{W^{(2)}} \mathcal{L} = \frac{1}{m} A^{(1)\top} \delta^{(2)}, \quad \nabla_{b^{(2)}} \mathcal{L} = \frac{1}{m} \sum_i \delta_i^{(2)}$$

**Propagación al bloque oculto:**

$$\delta_{\text{lin}}^{(1)} = \delta^{(2)} W^{(2)\top}, \quad (m \times n_h)$$

$$\delta^{(1)} = \delta_{\text{lin}}^{(1)} \odot \text{ReLU}'(Z^{(1)}) \quad \text{donde } \text{ReLU}'(z) = 1[z > 0]$$

$$\nabla_{W^{(1)}} \mathcal{L} = \frac{1}{m} X^\top \delta^{(1)}, \quad \nabla_{b^{(1)}} \mathcal{L} = \frac{1}{m} \sum_i \delta_i^{(1)}$$



## Paso 4: Retropropagación — Ecuaciones

**Recordar:** La retropropagación usa la regla de la cadena para calcular  $\nabla \mathcal{L}$  respecto a cada parámetro.

**Capa de salida:**

$$\delta^{(2)} = \hat{Y} - Y, \quad (m \times 1)$$

$$\nabla_{W^{(2)}} \mathcal{L} = \frac{1}{m} A^{(1)\top} \delta^{(2)}, \quad \nabla_{b^{(2)}} \mathcal{L} = \frac{1}{m} \sum_i \delta_i^{(2)}$$

**Propagación al bloque oculto:**

$$\delta_{\text{lin}}^{(1)} = \delta^{(2)} W^{(2)\top}, \quad (m \times n_h)$$

$$\delta^{(1)} = \delta_{\text{lin}}^{(1)} \odot \text{ReLU}'(Z^{(1)}) \quad \text{donde } \text{ReLU}'(z) = 1[z > 0]$$

$$\nabla_{W^{(1)}} \mathcal{L} = \frac{1}{m} X^\top \delta^{(1)}, \quad \nabla_{b^{(1)}} \mathcal{L} = \frac{1}{m} \sum_i \delta_i^{(1)}$$



## Paso 4: Retropropagación — Cálculo numérico

$$\delta^{(2)} = \hat{Y} - Y:$$

$$\delta^{(2)} = [0,522, 0,532, 0,515, 0,545, 0,533, -0,416, -0,403, -0,426, -0,414, -0,402]^T$$

Las clases 1 tienen gradiente **negativo**: la red predijo muy poco (0.58 en vez de 1.0).



## Paso 4: Retropropagación — Cálculo numérico

$$\delta^{(2)} = \hat{Y} - Y:$$

$$\delta^{(2)} = [0,522, 0,532, 0,515, 0,545, 0,533, -0,416, -0,403, -0,426, -0,414, -0,402]^T$$

Las clases 1 tienen gradiente **negativo**: la red predijo muy poco (0.58 en vez de 1.0).

**Gradientes de la capa de salida:**

$$\nabla_{W^{(2)}} \mathcal{L} = \frac{1}{10} A^{(1)\top} \delta^{(2)} = \begin{bmatrix} -0,009 \\ -0,088 \end{bmatrix} \quad \nabla_{b^{(2)}} \mathcal{L} = \frac{1}{10} \sum_i \delta_i^{(2)} = +0,059$$



## Paso 4: Retropropagación — Cálculo numérico

$$\delta^{(2)} = \hat{Y} - Y:$$

$$\delta^{(2)} = [0,522, 0,532, 0,515, 0,545, 0,533, -0,416, -0,403, -0,426, -0,414, -0,402]^T$$

Las clases 1 tienen gradiente **negativo**: la red predijo muy poco (0.58 en vez de 1.0).

**Gradientes de la capa de salida:**

$$\nabla_{W^{(2)}} \mathcal{L} = \frac{1}{10} A^{(1)\top} \delta^{(2)} = \begin{bmatrix} -0,009 \\ -0,088 \end{bmatrix} \quad \nabla_{b^{(2)}} \mathcal{L} = \frac{1}{10} \sum_i \delta_i^{(2)} = +0,059$$

**Gradientes de la capa oculta:** ( $\delta^{(1)}$  incluye la máscara de ReLU):

$$\nabla_{W^{(1)}} \mathcal{L} \approx \begin{bmatrix} -0,063 & -0,040 \\ -0,064 & -0,039 \end{bmatrix} \quad \nabla_{b^{(1)}} \mathcal{L} \approx \begin{bmatrix} -0,003 & +0,024 \end{bmatrix}$$

*Nota: Los ejemplos  $i = 3, 5, 8$  tienen  $\delta_{h_1}^{(1)} = 0$  porque  $z_{h_1}^{(1)} < 0 \Rightarrow$  ReLU bloquea el gradiente (problema del “neurón muerto”).*



## Paso 5: Actualización de Pesos

Regla de descenso de gradiente:  $\theta \leftarrow \theta - \alpha \cdot \nabla_{\theta} \mathcal{L}$ , con  $\alpha = 0,5$

Capa de salida:

$$W_{\text{new}}^{(2)} = \begin{bmatrix} 0,7 \\ 0,4 \end{bmatrix} - 0,5 \cdot \begin{bmatrix} -0,009 \\ -0,088 \end{bmatrix} = \begin{bmatrix} 0,705 \\ 0,444 \end{bmatrix}$$

$$b_{\text{new}}^{(2)} = 0,0 - 0,5 \cdot (0,059) = -0,030$$



## Paso 5: Actualización de Pesos

Regla de descenso de gradiente:  $\theta \leftarrow \theta - \alpha \cdot \nabla_{\theta} \mathcal{L}$ , con  $\alpha = 0,5$

Capa de salida:

$$W_{\text{new}}^{(2)} = \begin{bmatrix} 0,7 \\ 0,4 \end{bmatrix} - 0,5 \cdot \begin{bmatrix} -0,009 \\ -0,088 \end{bmatrix} = \begin{bmatrix} 0,705 \\ 0,444 \end{bmatrix}$$

$$b_{\text{new}}^{(2)} = 0,0 - 0,5 \cdot (0,059) = -0,030$$

Capa oculta:

$$W_{\text{new}}^{(1)} = \begin{bmatrix} 0,5 & 0,3 \\ -0,4 & 0,6 \end{bmatrix} - 0,5 \cdot \begin{bmatrix} -0,063 & -0,040 \\ -0,064 & -0,039 \end{bmatrix} = \begin{bmatrix} 0,532 & 0,320 \\ -0,368 & 0,620 \end{bmatrix}$$

$$b_{\text{new}}^{(1)} = [0,0, 0,0] - 0,5 \cdot [-0,003, 0,024] = [0,002, -0,012]$$



## Paso 5: Actualización de Pesos

Regla de descenso de gradiente:  $\theta \leftarrow \theta - \alpha \cdot \nabla_{\theta} \mathcal{L}$ , con  $\alpha = 0,5$

Capa de salida:

$$W_{\text{new}}^{(2)} = \begin{bmatrix} 0,7 \\ 0,4 \end{bmatrix} - 0,5 \cdot \begin{bmatrix} -0,009 \\ -0,088 \end{bmatrix} = \begin{bmatrix} 0,705 \\ 0,444 \end{bmatrix}$$

$$b_{\text{new}}^{(2)} = 0,0 - 0,5 \cdot (0,059) = -0,030$$

Capa oculta:

$$W_{\text{new}}^{(1)} = \begin{bmatrix} 0,5 & 0,3 \\ -0,4 & 0,6 \end{bmatrix} - 0,5 \cdot \begin{bmatrix} -0,063 & -0,040 \\ -0,064 & -0,039 \end{bmatrix} = \begin{bmatrix} 0,532 & 0,320 \\ -0,368 & 0,620 \end{bmatrix}$$

$$b_{\text{new}}^{(1)} = [0,0, 0,0] - 0,5 \cdot [-0,003, 0,024] = [0,002, -0,012]$$

**Interpretación:** Todos los pesos de  $W^{(1)}$  y  $W^{(2)}$  crecen levemente en magnitud (los gradientes negativos se invierten). Después de **múltiples épocas**, la loss disminuye y el modelo aprende la separación entre clases.



## Paso 6: Evaluación en Test Set

**Regla:** En test, se hace solo **forward pass** con los pesos entrenados. **No** se calculan ni actualizan gradientes.



## Paso 6: Evaluación en Test Set

**Regla:** En test, se hace solo **forward pass** con los pesos entrenados. **No** se calculan ni actualizan gradientes.

**Con pesos iniciales** (época 0) — para ilustrar el proceso:

| $i$ | $x_1$ | $x_2$ | $y$ | $z^{(2)}$ | $\hat{y}$ | $\hat{y}_{\text{pred}}$ |
|-----|-------|-------|-----|-----------|-----------|-------------------------|
| 1   | 0.20  | 0.30  | 0   | 0.096     | 0.524     | 1 ✗                     |
| 2   | 0.40  | 0.10  | 0   | 0.184     | 0.546     | 1 ✗                     |
| 3   | 0.70  | 0.80  | 1   | 0.297     | 0.574     | 1 ✓                     |
| 4   | 0.90  | 0.70  | 1   | 0.395     | 0.597     | 1 ✓                     |



## Paso 6: Evaluación en Test Set

**Regla:** En test, se hace solo **forward pass** con los pesos entrenados. **No** se calculan ni actualizan gradientes.

**Con pesos iniciales** (época 0) — para ilustrar el proceso:

| $i$ | $x_1$ | $x_2$ | $y$ | $z^{(2)}$ | $\hat{y}$ | $\hat{y}_{\text{pred}}$ |
|-----|-------|-------|-----|-----------|-----------|-------------------------|
| 1   | 0.20  | 0.30  | 0   | 0.096     | 0.524     | 1 ✗                     |
| 2   | 0.40  | 0.10  | 0   | 0.184     | 0.546     | 1 ✗                     |
| 3   | 0.70  | 0.80  | 1   | 0.297     | 0.574     | 1 ✓                     |
| 4   | 0.90  | 0.70  | 1   | 0.395     | 0.597     | 1 ✓                     |

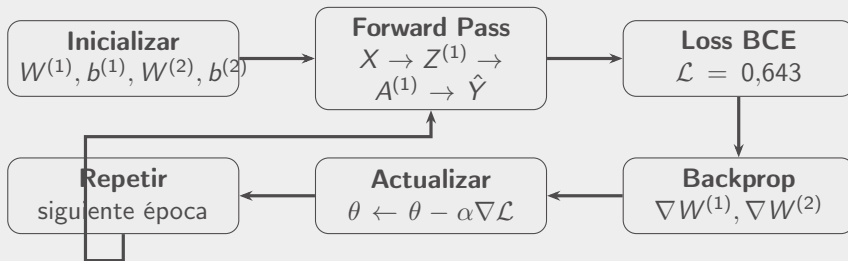
$$\text{Accuracy}_{\text{inicial}} = \frac{2}{4} = 50\% \quad \leftarrow \text{¡Como predicción aleatoria!}$$

**Después de entrenar 1000 épocas** (con PyTorch, ver notebook):

$$\text{Accuracy}_{\text{entrenado}} = \frac{4}{4} = 100\%$$



# Resumen Visual: Un Epoch Completo



**Criterios de parada:** número fijo de épocas, pérdida por debajo de umbral, o validación sin mejora (early stopping).



# Parte 3

Ejercicio propuesto para resolver



## Ejercicio 2: Clasificación de Crédito

**Problema:** Predecir si un cliente tendrá **mora** ( $y = 1$ ) o **no** ( $y = 0$ ) basándose en dos indicadores normalizados:  $x_1$  = ratio de deuda,  $x_2$  = historial crediticio inverso.

**Dataset de Test** (5 ejemplos):

**Dataset de Entrenamiento** (20 ejemplos):

| $i$ | $x_1$ | $x_2$ | $y$ | $i$ | $x_1$ | $x_2$ | $y$ |
|-----|-------|-------|-----|-----|-------|-------|-----|
| 1   | 0.15  | 0.20  | 0   | 11  | 0.65  | 0.70  | 1   |
| 2   | 0.25  | 0.10  | 0   | 12  | 0.75  | 0.80  | 1   |
| 3   | 0.30  | 0.35  | 0   | 13  | 0.80  | 0.70  | 1   |
| 4   | 0.10  | 0.30  | 0   | 14  | 0.70  | 0.85  | 1   |
| 5   | 0.40  | 0.15  | 0   | 15  | 0.85  | 0.75  | 1   |
| 6   | 0.20  | 0.45  | 0   | 16  | 0.60  | 0.90  | 1   |
| 7   | 0.35  | 0.25  | 0   | 17  | 0.90  | 0.65  | 1   |
| 8   | 0.45  | 0.40  | 0   | 18  | 0.75  | 0.65  | 1   |
| 9   | 0.15  | 0.40  | 0   | 19  | 0.60  | 0.80  | 1   |
| 10  | 0.50  | 0.30  | 0   | 20  | 0.80  | 0.90  | 1   |

| $i$ | $x_1$ | $x_2$ | $y$ |
|-----|-------|-------|-----|
| 1   | 0.20  | 0.30  | 0   |
| 2   | 0.45  | 0.25  | 0   |
| 3   | 0.35  | 0.45  | 0   |
| 4   | 0.70  | 0.75  | 1   |
| 5   | 0.85  | 0.80  | 1   |

**Arquitectura:**  $2 \rightarrow 3 \rightarrow 1$   
(3 neuronas ocultas con ReLU)



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.





## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.
3. Calcula el forward pass para los ejemplos  $i = 1$  (clase 0) e  $i = 11$  (clase 1). Muestra todos los pasos intermedios.



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.
3. Calcula el forward pass para los ejemplos  $i = 1$  (clase 0) e  $i = 11$  (clase 1). Muestra todos los pasos intermedios.
4. Calcula la matriz  $Z^{(1)}$  completa ( $20 \times 3$ ) y aplica ReLU para obtener  $A^{(1)}$ .  
¿Cuántos valores son cortados por ReLU?



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.
3. Calcula el forward pass para los ejemplos  $i = 1$  (clase 0) e  $i = 11$  (clase 1). Muestra todos los pasos intermedios.
4. Calcula la matriz  $Z^{(1)}$  completa ( $20 \times 3$ ) y aplica ReLU para obtener  $A^{(1)}$ .  
¿Cuántos valores son cortados por ReLU?
5. Calcula  $\hat{Y}$  ( $20 \times 1$ ) y la pérdida BCE  $\mathcal{L}$ .



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.
3. Calcula el forward pass para los ejemplos  $i = 1$  (clase 0) e  $i = 11$  (clase 1). Muestra todos los pasos intermedios.
4. Calcula la matriz  $Z^{(1)}$  completa ( $20 \times 3$ ) y aplica ReLU para obtener  $A^{(1)}$ .  
¿Cuántos valores son cortados por ReLU?
5. Calcula  $\hat{Y}$  ( $20 \times 1$ ) y la pérdida BCE  $\mathcal{L}$ .
6. Calcula los gradientes  $\nabla_{W^{(1)}} \mathcal{L}$ ,  $\nabla_{b^{(1)}} \mathcal{L}$ ,  $\nabla_{W^{(2)}} \mathcal{L}$ ,  $\nabla_{b^{(2)}} \mathcal{L}$ .



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.
3. Calcula el forward pass para los ejemplos  $i = 1$  (clase 0) e  $i = 11$  (clase 1). Muestra todos los pasos intermedios.
4. Calcula la matriz  $Z^{(1)}$  completa ( $20 \times 3$ ) y aplica ReLU para obtener  $A^{(1)}$ .  
¿Cuántos valores son cortados por ReLU?
5. Calcula  $\hat{Y}$  ( $20 \times 1$ ) y la pérdida BCE  $\mathcal{L}$ .
6. Calcula los gradientes  $\nabla_{W^{(1)}} \mathcal{L}$ ,  $\nabla_{b^{(1)}} \mathcal{L}$ ,  $\nabla_{W^{(2)}} \mathcal{L}$ ,  $\nabla_{b^{(2)}} \mathcal{L}$ .
7. Actualiza los pesos con  $\alpha = 0,1$ .



## Ejercicio 2: Tareas

Resolver manualmente (1 época) y luego implementar en PyTorch:

1. Dibuja el diagrama de la red  $2 \rightarrow 3 \rightarrow 1$  indicando todos los parámetros.
2. Inicializa  $W^{(1)} \in \mathbb{R}^{2 \times 3}$ ,  $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ,  $W^{(2)} \in \mathbb{R}^{3 \times 1}$ ,  $b^{(2)} \in \mathbb{R}^{1 \times 1}$  con valores pequeños de tu elección.
3. Calcula el forward pass para los ejemplos  $i = 1$  (clase 0) e  $i = 11$  (clase 1). Muestra todos los pasos intermedios.
4. Calcula la matriz  $Z^{(1)}$  completa ( $20 \times 3$ ) y aplica ReLU para obtener  $A^{(1)}$ .  
¿Cuántos valores son cortados por ReLU?
5. Calcula  $\hat{Y}$  ( $20 \times 1$ ) y la pérdida BCE  $\mathcal{L}$ .
6. Calcula los gradientes  $\nabla_{W^{(1)}} \mathcal{L}$ ,  $\nabla_{b^{(1)}} \mathcal{L}$ ,  $\nabla_{W^{(2)}} \mathcal{L}$ ,  $\nabla_{b^{(2)}} \mathcal{L}$ .
7. Actualiza los pesos con  $\alpha = 0,1$ .
8. Evalúa el modelo en el test set: calcula  $\hat{Y}_{\text{test}}$  y la accuracy inicial y final (después de entrenar en PyTorch).